

BLOCKCHAIN LABORATORY

Experiment No. 1

Cryptography in Blockchain, Merkle Root Tree Hash

Name: Nikhil Sunchu

Roll No.: 31

Class/section D20A

Date _____

1. Aim

Cryptography in Blockchain, Merkle Root Tree Hash.

2. Lab Objectives

To explore Blockchain concepts.

2.1. Lab Outcome

LO1: Creating Cryptographic Hash using Merkle Tree.

3. Tasks to be Performed

1. Make a copy of the Google Colab Notebook.
2. Try to solve the errors in each of the four programs.
3. In the fourth program, *Constructing a Merkle Tree Root Hash*, modify the code as follows:
 - Update the transactions list with valid entries.

Example:

```
1 transactions = [  
2     'A -> B : $10',  
3     'B -> C : $5',  
4     'C -> A : $2'  
5 ]
```

4. Sample transactions to be considered:
 - T1: Alice $\rightarrow$ Bob : \$200
 - T2: Bob $\rightarrow$ Dave : \$500
 - T3: Dave $\rightarrow$ Eve : \$100
 - T4: Eve $\rightarrow$ Alice : \$300
 - T5: Roo $\rightarrow$ Bob : \$50
5. Hash the transactions before combining them in the **for** loop.
6. Print all the intermediate hashes during the construction of the Merkle Tree Root Hash.

4. Tools and Libraries Used

- Python
- Python library: hashlib

5. Tasks Performed

1. Hash Generation using SHA-256:

- Developed a Python program to compute a SHA-256 hash for any given input string using the hashlib library.

2. Target Hash Generation with Nonce:

- Created a program to generate a hash code by concatenating a user input string and a nonce value to simulate the mining process.

3. Proof-of-Work Puzzle Solving:

- Implemented a program to find the nonce that, when combined with a given input string, produces a hash starting with a specified number of leading zeros.

4. Merkle Tree Construction:

- Built a Merkle Tree from a list of transactions by recursively hashing pairs of transaction hashes, doubling the last node if needed, and generating the Merkle Root hash for blockchain transaction integrity.

6. Programs and Outputs

6.1. Program 1: SHA-256 Hash Generation

```
1 import hashlib
2
3 def create_hash(string):
4     # Create a hash object using SHA-256 algorithm
5     hash_object = hashlib.sha256()
6
7     # Convert the string to bytes and update the hash object
8     hash_object.update(string.encode('utf-8'))
9
10    # Get the hexadecimal representation of the hash
11    hash_string = hash_object.hexdigest()
12
```

```
13     # Return the hash string
14     return hash_string
15
16
17 # Example usage
18 input_string = input("Enter a string: ")
19 hash_result = create_hash(input_string)
20 print("Hash:", hash_result)
```

Output:

```
Enter a string: NikhilSunchu
Hash: 4758bd001bb84cf08b1010df52aaf02b2d1fa28eb7da5077fc5391d4727b206d
```

6.2. Program 2: Target Hash Generation with Nonce

```
1 import hashlib
2
3 # Get user input
4 input_string = input("Enter a string: ")
5 nonce = input("Enter the nonce: ")
6
7 # Concatenate the string and nonce
8 hash_string = input_string + nonce
9
10 # Calculate the hash using SHA-256
11 hash_object = hashlib.sha256(hash_string.encode('utf-8'))
12 hash_code = hash_object.hexdigest()
13
14 # Print the hash code
15 print("Hash Code:", hash_code)
```

Output:

```
Enter a string: NikhilSunchu
Enter the nonce: 31
Hash Code:
aed188a474ce54614e5b9224ca8956a559707e69aaaa3ccae58e49e1f557e740
```

6.3. Program 3: Proof-of-Work Puzzle Solving

The following is the program presented in the source document, with its indentation formatted for readability.

```
1 import hashlib
2
3 def find_nonce(input_string, num_zeros):
4     nonce = 0
5     hash_prefix = '0' * num_zeros
6
7     while True:
8         # Concatenate the string and nonce
9         hash_string = input_string + str(nonce)
10
11         # Calculate the hash using SHA-256
12         hash_object = hashlib.sha256(hash_string.encode('utf-8'))
13         hash_code = hash_object.hexdigest()
14
15         # Check if the hash code has the required number of leading zeros
16         if hash_code.startswith(hash_prefix):
17             print("Hash:", hash_code)
18             return nonce
19
20     nonce += 1
21
22
23 # Get user input
24 input_string = "NikhilSunchu"
25 num_zeros = 1
26
27 # Find the expected nonce
28 expected_nonce = find_nonce(input_string, num_zeros)
29
30 # Print the expected nonce
31 print("Input String:", input_string)
32 print("Leading Zeros:", num_zeros)
33 print("Expected Nonce:", expected_nonce)
```

Output:

```
Hash: 02ea712faf82d8f28cbf209efce1276ee29628da245879c3b569daf6a3cf370c
Input String: NikhilSunchu
Leading Zeros: 1
Expected Nonce: 11
```

6.4. Program 4: Merkle Tree Construction

```
1 import hashlib
2
3 def hash_data(data):
4     return hashlib.sha256(data.encode('utf-8')).hexdigest()
5
6 def build_merkle_tree(transactions):
7     if len(transactions) == 0:
8         return None
9
10    transactions = [hash_data(transaction) for transaction in transactions]
11
12    print("Initial Transaction Hashes: ")
13
14    for i, transaction in enumerate(transactions):
15        print(f"Transaction {i+1}: {transaction}")
16
17    while len(transactions) > 1:
18
19        if len(transactions) % 2 != 0:
20            transactions.append(transactions[-1])
21
22        new_transactions = []
23
24        for i in range(0, len(transactions), 2):
25            combined = transactions[i] + transactions[i+1]
26            hash_combined = hashlib.sha256(combined.encode('utf-8')).hexdigest
27            ()
28
29            print(f"Hash {i // 2 + 1}")
30            print(f"{transactions[i]}")
31            print(f" + {transactions[i+1]}")
32            print(f" = {hash_combined}")
33
34            new_transactions.append(hash_combined)
35
36        transactions = new_transactions
37
38    return transactions[0]
39
40 # Example usage
```

```
40 transactions = ["Alice → Bob : $200", "Bob → Dave : $500", "Dave → Eve :  
    $100", "Eve → Alice : $300", "Roo → Bob : $50"]  
41  
42 merkle_root = build_merkle_tree(transactions)  
43 print("Merkle Root:", merkle_root)
```

Output:

Initial Transaction Hashes:

Transaction 1:

d7d1c2448f818062ab9bb89aa9f95564670895f927d730d2d43ba8e5cada8a4d

Transaction 2:

ebc396058242fc4cb711aa1dd2c2ce1c44818b9d35c9d53465bee3c12462072e

Transaction 3: 56

a42d8ba343cbcf8133e8a0300564876f2e76dfe8281db52fccbad24fc8d88a

Transaction 4:

fc3d2297f8b8aa145fa4ded66a1ed06db9d46d661a8d3673e1ceb0dbbaf597a6

Transaction 5: 70

ec9a2f7f21cf62ef7acfd60032bcc094a2739d2ae73dea3e929512b9a00c4a

Hash 1

d7d1c2448f818062ab9bb89aa9f95564670895f927d730d2d43ba8e5cada8a4d
+ ebc396058242fc4cb711aa1dd2c2ce1c44818b9d35c9d53465bee3c12462072e
= 51215ac8c39b7d8d72ce873c5cf75a9b7d3edd069c79007a1f0827af3e43d258

Hash 2

56a42d8ba343cbcf8133e8a0300564876f2e76dfe8281db52fccbad24fc8d88a
+ fc3d2297f8b8aa145fa4ded66a1ed06db9d46d661a8d3673e1ceb0dbbaf597a6
= b5e0a195812483b212784056d2bf15f8caeff4fb4bde6e4910d9827d039632f1

Hash 3

70ec9a2f7f21cf62ef7acfd60032bcc094a2739d2ae73dea3e929512b9a00c4a
+ 70ec9a2f7f21cf62ef7acfd60032bcc094a2739d2ae73dea3e929512b9a00c4a
= 7b5c2d1c9a72494009e89c8608db836a79e71e13f5d7f12822b026364a22b5b6

Hash 1

51215ac8c39b7d8d72ce873c5cf75a9b7d3edd069c79007a1f0827af3e43d258
+ b5e0a195812483b212784056d2bf15f8caeff4fb4bde6e4910d9827d039632f1
= 8e3aa833a93cf5663d5138a8d80d8e6a8cf006517a462fb4260806795701f45d

Hash 2

7b5c2d1c9a72494009e89c8608db836a79e71e13f5d7f12822b026364a22b5b6
+ 7b5c2d1c9a72494009e89c8608db836a79e71e13f5d7f12822b026364a22b5b6
= 5b0369cfb2de9ed3b41d380675c240323de05bc9b9062fae8cd5991902828cd2

Hash 1

8e3aa833a93cf5663d5138a8d80d8e6a8cf006517a462fb4260806795701f45d
+ 5b0369cfb2de9ed3b41d380675c240323de05bc9b9062fae8cd5991902828cd2

```
= 382f188ce8666903c084f5704ade4a7e1a1948242bdfd0de2b529d9fedb6fe2f  
Merkle Root: 382f188ce8666903c084f5704ade4a7e1a1948242bdfd0de2b529d9fedb6fe2f
```

Note: The source document instructs that the transactions should be hashed before they are combined in the `for` loop and that intermediate hashes should be printed. The output above is the output recorded in the source document; the program listing has been formatted without changing that recorded result.

7. Viva Preparation

7.1. 1. Cryptographic Hash Functions in Blockchain

A cryptographic hash function is used in blockchain to generate a fixed-length representation of input data. SHA-256 is used in the programs in this experiment.

7.2. 2. What is a Merkle Tree?

A Merkle Tree is a tree-like data structure in which transaction data is represented by hashes. Pairs of hashes are combined and hashed repeatedly until a single hash, called the Merkle Root, is obtained.

7.3. 3. What is a Cryptographic Puzzle and Explain the Golden Nonce

A cryptographic puzzle requires finding a nonce that produces a hash satisfying a specified condition. In the experiment, the condition is that the resulting hash begins with a specified number of leading zeros. The nonce satisfying the required condition is the expected or “golden” nonce.

7.4. 4. How Does a Merkle Tree Work?

The transactions are converted into hashes. Hashes are combined in pairs and hashed again. If the number of nodes at a level is odd, the last node is duplicated. This process continues until one hash remains, which is the Merkle Root.

7.5. 5. Benefits of a Merkle Tree

- Efficient representation of a large number of transactions.
- Enables efficient verification of transaction integrity.
- A single Merkle Root summarizes the transaction set.
- Supports fast verification of data in blockchain blocks.

7.6. 6. Use Cases of a Merkle Tree

Merkle Trees are used in blockchain systems to summarize transactions and verify that transaction data has not been altered. They are particularly useful for efficient transaction verification.

8. Conclusion

This study demonstrates fundamental cryptographic concepts and their applications within blockchain technology:

- **SHA-256 Hashing** provides a secure and unique fingerprint for any input, ensuring data integrity.
- **Nonce and Target Hashing** allow the simulation of proof-of-work consensus, emphasizing the difficulty of mining blocks by meeting hash conditions such as leading zeros.
- **Proof-of-Work Puzzle Solving** shows how mining requires computational effort to find a nonce that makes the hash satisfy the required network difficulty.
- **Merkle Tree Construction** efficiently summarizes and validates large numbers of transactions with a single Merkle Root hash, which is essential in blockchain for fast verification and ensuring the integrity of data blocks.

Together, these implementations capture the essence of how cryptography underpins blockchain security, immutability, and efficiency, especially through hash functions and structures such as Merkle Trees that enable trustless and decentralized transaction management.